UART-16550 IP with APB interface

IP implemented:

I have implemented the OpenCores UART16550 — a Universal Asynchronous Receiver Transmitter that is fully compatible with the industry standard National Semiconductor NS16550A chip.

Core features of the IP:

Serial Communication:

- Transmits and receives data serially over a single wire
- Supports standard RS232 protocol
- Supports both Half duplex and full duplex communication
- Configurable data format — 5 to 8 data bits, 1 or 2 stop bits
- Configurable parity — odd, even, or none

FIFO Buffers:

- 16-byte TX FIFO — holds data waiting to be transmitted
- 16-byte RX FIFO — holds received data waiting to be read
- Programmable RX trigger levels — 1, 4, 8 or 14 bytes

Baud Rate Generation:

- Fully programmable via 16-bit Divisor Latch
- Formula: Baud Rate = Clock / (16 × DL)
- Your project: DL=54 at 100MHz → 115,200 baud

Interrupt System:

- Full priority interrupt chain via IIR register
- RX Line Status, RX Data Available, Timeout, THRE, Modem Status
- All maskable via IER register

Modem Signals:

- Full modem signal support — RTS, CTS, DTR, DSR, RI, DCD
- Hardware flow control
- Internal loopback mode for self-test

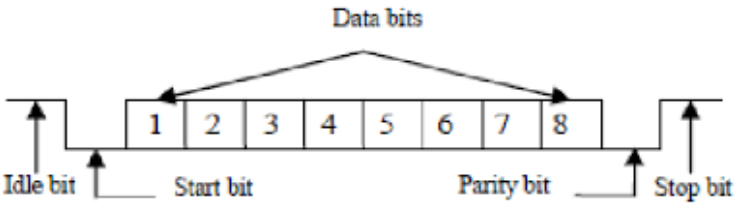
WORKING OVERVIEW

APB Protocol (Master Interface)

• Protocol Overview:

-Used for low-bandwidth register access.
• Write Transfer: –
-Setup: PSEL=1, PENABLE=0.
-Access: PENABLE=1. Data latched when PREADY=1.
• Wait States: –
-Slave holds PREADY Low to extend transfer.

UART Character frame:



- Start bit- logic 0 (synchronizes receiver)
- Data bits- 5-8 bits(LSB first)
- Parity bit- optional(odd/even)
- Stop bit- Logic 1 (end of frame)

TRANSMISSION- simple flow:

1. CPU sends data

- Writes a byte via APB

2. UART picks data

- When transmitter is free

3. Frame is created

- Start bit (0)
- Data bits
- Stop bit (1)

4. Bits are sent serially

- One by one on TX line
- Based on baud rate timing

5.. Repeat

- Next byte from FIFO is sent

RECEPTION – simple flow:

1.Line is idle (HIGH)

2.Start bit detected

- Line goes HIGH → LOW

3.UART starts reading bits

- Samples incoming bits at correct timing

4.Byte is formed

- Bits collected into one byte

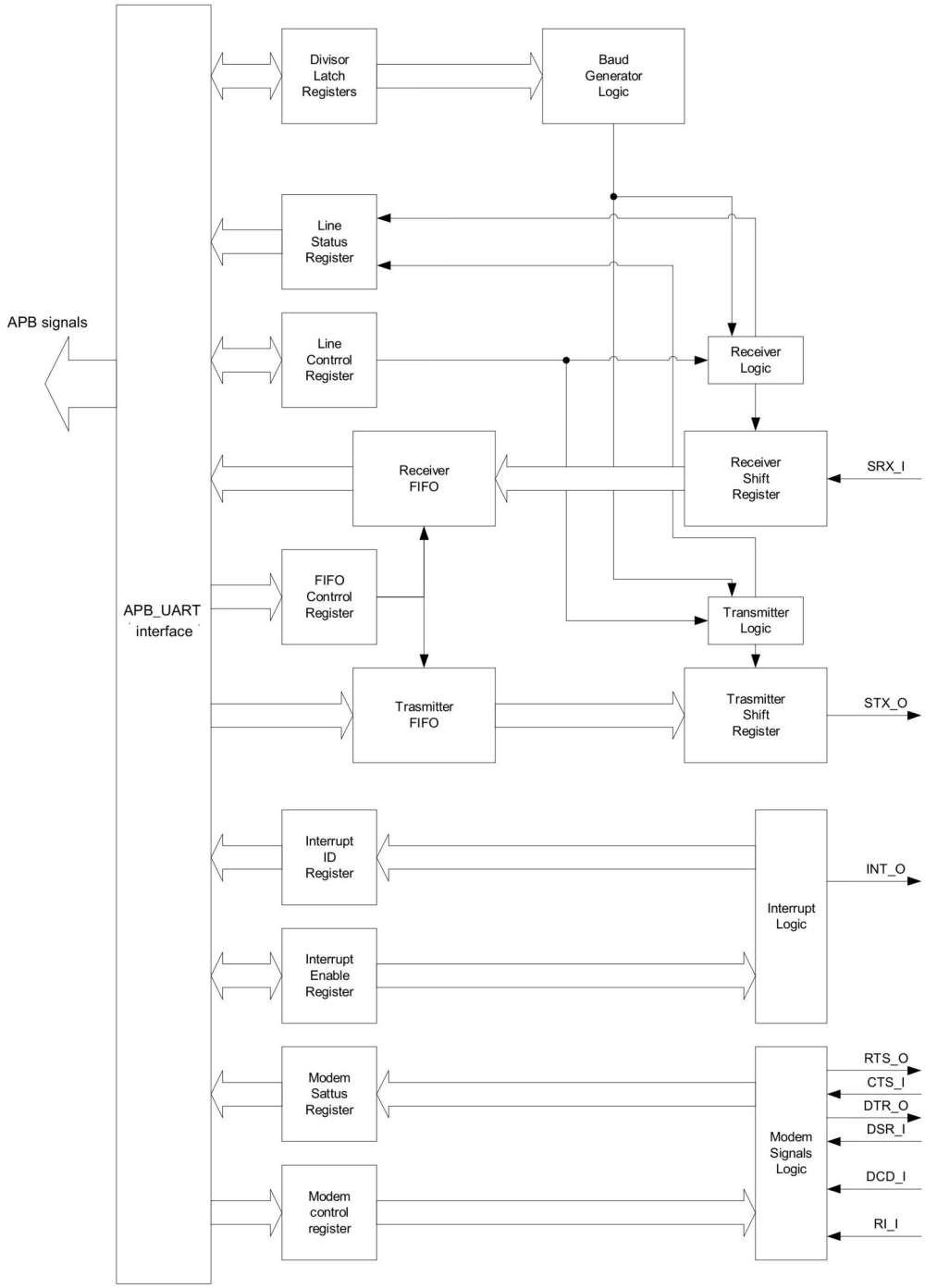
5.CPU reads data

- Reads byte via APB

6.Repeat

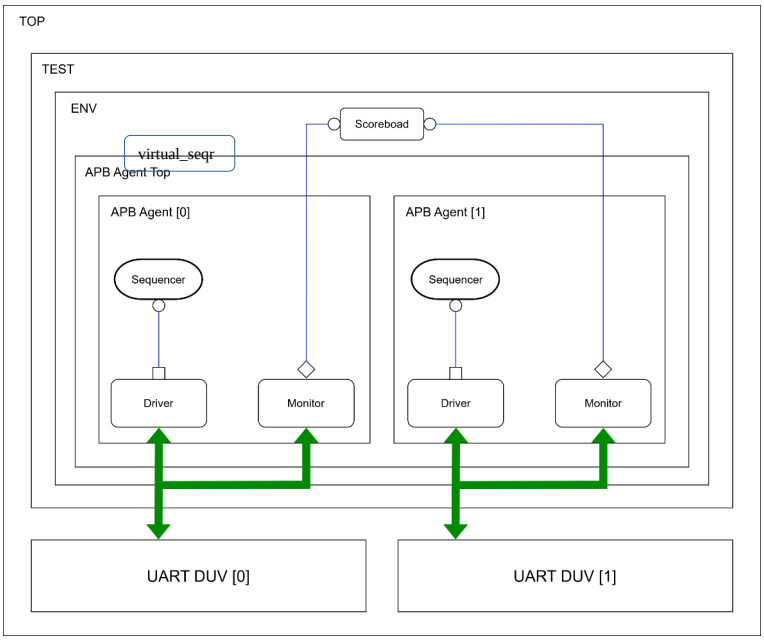
- For next incoming data

Final Block Diagram:

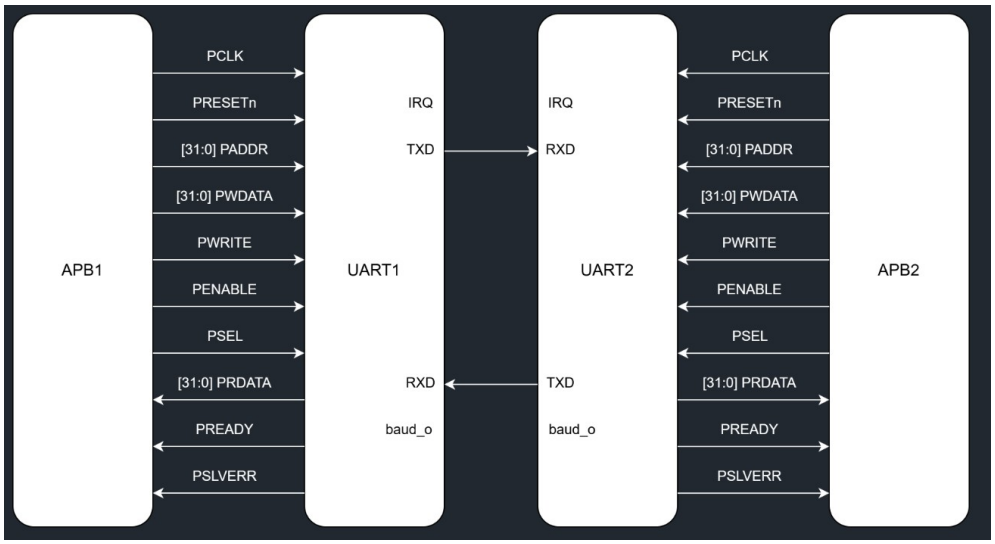


VERIFICATION

UVM TB architecture



DUV Block diagram:



Testcases used:

1. Half-duplex test

Only one DUT transmits at a time — the other only receives

- DUT1 is configured and sends data byte 0x07 via THR register
- DUT2 is configured with same baud rate and waits to receive
- After transmission DUT2 reads IIR to check interrupt type and reads RBR to get received data
- Scoreboard checks that DUT1 transmitted data matches DUT2 received data
- Verifies one directional serial communication works correctly

2.Full-duplex test
Both DUT1 and DUT2 transmit and receive simultaneously

- DUT1 sends 0x07 and DUT2 sends 0x0A at the same time
- Both DUTs are configured with same baud rate, data format and FIFO settings
- Each DUT monitors its own interrupt and reads received data
- Scoreboard cross checks DUT1 TX data equals DUT2 RX data and DUT2 TX data equals DUT1 RX data
- Verifies simultaneous bidirectional serial communication

3.Loop back test
MCR register bit 4 is set to 1 on both DUTs to enable internal loopback mode

- In loopback mode the TX output is internally connected to the RX input inside the same UART
- DUT1 sends 0x07 and DUT2 sends 0x0A — each DUT receives its own transmitted data
- No external serial connection is needed between DUTs for this test
- Scoreboard checks that each DUT received exactly what it transmitted
- Verifies the internal TX to RX path of each UART works correctly

4. Parity error test
DUT1 is configured with even parity by setting LCR = 0x1B

- DUT2 is configured with odd parity by setting LCR = 0x0B
- When DUT1 transmits with even parity and DUT2 receives expecting odd parity a mismatch occurs
- The receiver detects the wrong parity bit and sets LSR bit 2 to 1
- IIR shows 0xC6 indicating Receiver Line Status interrupt fired
- Scoreboard verifies LSR[2] is set confirming parity error was correctly detected

5. Framing error test
DUT1 is configured with even parity by setting LCR = 0x1B

- DUT2 is configured with odd parity by setting LCR = 0x0B
- When DUT1 transmits with even parity and DUT2 receives expecting odd parity a mismatch occurs
- The receiver detects the wrong parity bit and sets LSR bit 2 to 1
- IIR shows 0xC6 indicating Receiver Line Status interrupt fired
- Scoreboard verifies LSR[2] is set confirming parity error was correctly detected

6. Overrun error test
DUT2 sends 17 random bytes continuously into DUT1 using two repeat blocks of 14 and 3 bytes

- DUT1 is configured but deliberately does not read the RX FIFO to allow it to fill
- When the 17th byte arrives the 16 byte RX FIFO is already full and the new byte is lost
- uart_rfifo.v detects the overflow and sets LSR bit 1 to 1
- IIR shows RLS interrupt with IIR bits 3 to 1 equal to 011
- Scoreboard verifies LSR[1] is set confirming overrun error was correctly detected

7. Timeout error test
DUT2 FCR is configured with trigger level 14 by setting FCR = 0xC6 meaning interrupt fires only when 14 bytes arrive

- DUT2 sends only 1 byte which is below the trigger level so the RDA interrupt never fires
- After 4 character times with no new data the UART generates a character timeout interrupt
- IIR shows 0xCC indicating timeout interrupt with bits 3 to 1 equal to 110
- This tests the case where data arrives in the FIFO but never reaches the trigger level
- Scoreboard verifies the timeout interrupt fired correctly

8. THR empty test

IER bit 1 is set to 1 on DUT1 to enable the THRE interrupt

- DUT1 transmits one byte 0x07 by writing to the THR register
- After the TX FIFO empties and the byte is fully transmitted the THRE interrupt fires
- IIR shows 0xC2 indicating THRE interrupt with bits 3 to 1 equal to 001
- Both DUT1 and DUT2 show IIR = 0xC2 confirming TX holding register is empty
- Scoreboard verifies the THRE interrupt fired correctly indicating the transmitter is ready for more data

NOTE: Information about working of UART registers is available in the specification file

What I have done:

- modified the wishbone interface to an APB compatible interface by mapping the wishbone signals to the corresponding APB signals
- Added new signal PSLVERR and implemented error logic to trigger PSLVERR
- Changed the design from active-high reset to active-low reset as APB follows active-low reset
- Successfully Implemented the verification plan and UVM verification environment
- verified the design with 8 test scenarios